

Implementation Report

Take Home Coding Exercise Maincode

Name:	Lakpa Tamang
Task:	Optimizing GPT2-like model
Dataset:	Hacker-news posts

Objective: Analyze and improve the training run of a small GPT-2 style model on a Hacker News headline dataset.

Summary: I was able to reduce the validation loss by approximately 22 times from the baseline of **1.754** to **0.08** within 7 epochs on HackerNews dataset.

1 Data Analysis

I started by doing some exploratory data analysis on the dataset. Since the baseline training was done only on the titles, I specifically plotted the distribution of titles column as a histogram plot as shown in Fig. 1. From this plot, I observed that the number of words in the titles vary from as less as 0 to more than 35. Quantitatively, the median value was found to be around 8, and over 96 % of the titles has at least 3 words and a maximum of 15 words in total. In the following sections, I explain how I utilize these metrics to train the model and improve the validation loss.

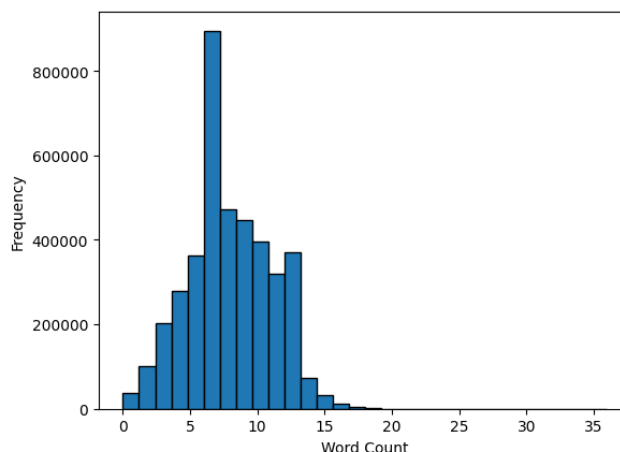


Figure 1: Histogram plot representing the distribution of titles in Hackernews dataset.

2 Changes Made

Throughout the implementation, I have made several changes that are discussed in a systematic manner below:

2.1 Tokenization

The first change I made was to update the old tokenizer. The updated tokenizer implementation brings several enhancements to data preprocessing and training consistency. One of the key changes made was to lowercase the text followed by appending an end-of-sequence `< eos >` token to the each title. This ensured the model learnt from a standardized corpus with clear sequence boundaries resulting in a cleaner vocabulary set and potentially better convergence for smaller models. Furthermore, I simplified the tokenizer wrapper by removing unused mappings, such as string-to-integer and vice versa which were apparently of no use. In addition, as mentioned in the data analysis section, a substantial portion of titles had a total word count between 3 and 15. Therefore, for training purpose I only used the data whose lengths are between 3 and 15. Specifically, I restricted the

sentences to this range, which potentially provides me two benefits: (i) minimization of the noise from brief, context-lacking phrases, and (ii) prevent excessively long sequences which may destabilize training.

2.2 Augmentation

Next change that I made was to use a data augmentation technique. As it is well known that data augmentation, if done properly can significantly enhance the generalization of a deep learning model (1). In this regard, I utilized a sliding window method (2) as a primary data augmentation technique which not only populates the existing corpus but also addresses short-context limitations in existing titles. The goal was to increase effective dataset size, and preserve context information for shorter sequences without discarding them. I set the context size for the sliding window technique to the median value of the length of titles i.e., 8 (see Fig. 1). The intuition behind this is to create a substantial size of meaningful contexts while also preserving the semantic characteristics of the token sequences. The overall procedure of sliding window technique is presented by Algorithm 1. This algorithm is guided by the different types of padding techniques that we can perform to create a context off a sequence. In my method, I set three different types of padding techniques: cyclic, edge, and token.

- *cyclic*: The sequence is repeated until the context length is reached.
- *edge*: The edge token of the sequence is repeated and appended to either beginning or end of the original sequence.
- *token*: A padding token $\langle \text{pad} \rangle$ is appended to either the beginning or the end of the original sequence.

Algorithm 2 represents the detailed heuristics of how the sequence are padded.

Algorithm 1 Sliding Window Technique

Require: list of titles T , context size C

Ensure: Fixed-length contexts list $contexts$

```

1:  $contexts \leftarrow []$ 
2: for all  $title \in T$  do
3:    $len \leftarrow \text{length}(title)$ 
4:   if  $len = 0$  then
5:     continue to next title
6:   else if  $len < C$  then
7:      $ctx \leftarrow \text{PadSequence}()$ 
8:     Append  $ctx$  (as string) to  $contexts$ 
9:   else
10:    for  $i = 0$  to  $len - C$  do
11:       $ctx \leftarrow words[i : i + C]$ 
12:      Append joined string  $ctx$  to  $contexts$ 
13:    end for
14:  end if
15: end for
16: return  $contexts$ 

```

Algorithm 2 PadSequence

Require: title t , context size C , pad mode p , align a , pad token tk

Ensure: Padded sequence Seq_{pad} of length C

```
1:  $L \leftarrow \text{length}(t)$ 
2:  $P \leftarrow C - L$ 
3: if  $P = 0$  then
4:   return first  $C$  tokens of  $t$ 
5: end if
6: if  $p = \text{"cyclic"}$  then
7:    $Seq_{pad} \leftarrow t$  repeated until  $\text{length}(Seq_{pad}) \geq C$ 
8:   return  $Seq_{pad}[:C]$ 
9: else if  $p = \text{"edge"}$  then
10:  if  $a = \text{"left"}$  then
11:     $Seq_{pad} \leftarrow [t[0]] \times P$ 
12:    return  $Seq_{pad} + t$ 
13:  else
14:     $Seq_{pad} \leftarrow [t[-1]] \times P$ 
15:    return  $t + Seq_{pad}$ 
16:  end if
17: else if  $p = \text{"token"}$  then
18:    $Seq_{pad} \leftarrow [tk] \times P$ 
19:   if  $a = \text{"left"}$  then
20:    return  $Seq_{pad} + t$ 
21:   else
22:    return  $t + Seq_{pad}$ 
23:   end if
24: end if
```

2.3 Optimization and Model Training

Next change was to update the optimization algorithm and training strategy. Here, I started by changing a few things. First, the optimizer was switched from SGD to AdamW (3), which decouples weight decay from the gradient updates, ensuring more effective regularization and better generalization. Second, a cosine learning rate scheduler with warmup was introduced in place of the previous CosineAnnealing scheduler. This combination allows for a smooth ramp-up in learning rate during the early warmup phase reducing instability in the first steps followed by a gradual, non-linear decay that encourages rapid initial learning and better convergence in later stages. These changes collectively enhances training stability, reduce overfitting, and help the model converge to flatter minima, ultimately improving performance compared to the original configuration.

Interesting thing to note here is that, I did not change model architecture, preserving the attention mechanism and original structure of the GPT-2. A noticeable change I did instead was to use consistency regularization (4) technique to optimize training of the existing model. In this technique, the prediction of current raw logits and the moving average logits are aligned to incur stable prediction across successive training steps. This was done to smoothen the learning curve. I trained the model with these configurations updated and plotted the validation curve relative to the epochs as shown in Fig. 2. Here, we can observe that the loss converges early and better than the baseline.

Consistency Regularization with Moving Average (MA) Logits. In this section, I show why the consistency regularization is important here and how does it impact the model performance. Let $\mathbf{z}_\tau \in \mathbb{R}^C$ denote the logits predicted by the model at the current iteration τ , where C is the context length. A moving average (MA) of the logits, denoted $\mathbf{z}_\tau^{\text{ma}}$, is calculated as:

$$\mathbf{z}_\tau^{\text{ma}} = \begin{cases} \mathbf{z}_\tau, & \text{if } \tau = 0, \\ \alpha \mathbf{z}_{\tau-1}^{\text{ma}} + (1 - \alpha) \mathbf{z}_\tau, & \text{if } \tau > 0, \end{cases} \quad (1)$$

where $\alpha \in [0, 1)$ is the MA decay coefficient. I set this value to 0.90, which means 90 % of past information was kept to update the moving average. By aligning current predictions with smoothed historical predictions, the model avoids abrupt, high-variance changes in its output distribution. A KL divergence (5) measure is used to align the prediction of these two entities:

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{i=1}^B D_{\text{KL}}(\text{softmax}(\mathbf{z}_i) \parallel \text{softmax}(\mathbf{z}_i^{\text{ema}})), \quad (2)$$

where B is the batch size and $D_{\text{KL}}(p||q) = \sum_{v=1}^V p_v \log \frac{p_v}{q_v}$.

The final training objective consists of the classification loss term ($\mathcal{L}_{\text{class}}$) along with the regularization λ_{KL} :

$$\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}}, \quad (3)$$

where λ_{KL} is a hyperparameter controlling the strength of consistency regularization. The value of λ_{KL} is set to 0.1 through brute force method.

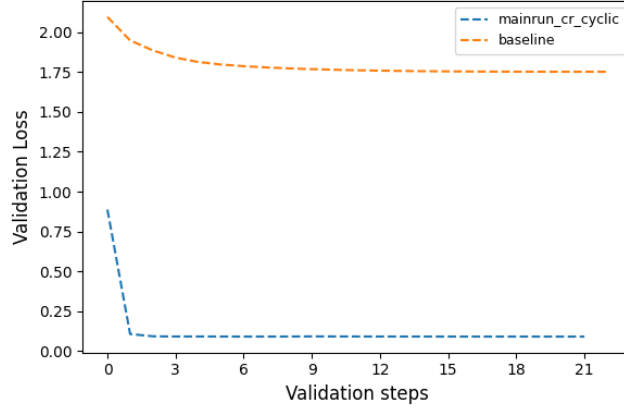


Figure 2: Comparison of the validation accuracy curve of proposed implementation with the baseline.

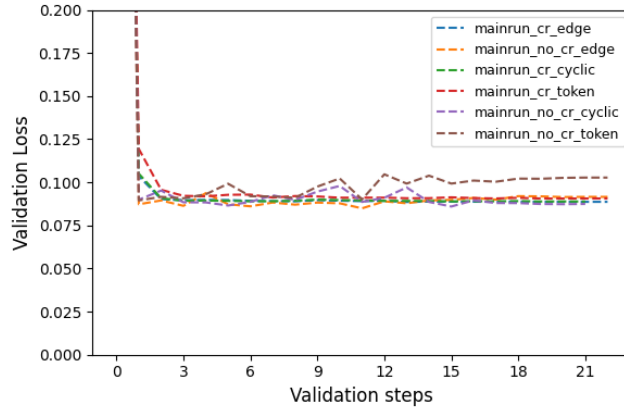


Figure 3: Comparison of validation learning curve of different padding techniques, each optimized with and without consistency regularization. cr represents consistency regularization.

We can see that in Fig. 3, through consistency regularization the loss becomes smoother with epochs. This is highly beneficial to prevent overfitting while scaling the problem.

References

- [1] R. Jha, C. Lovering, and E. Pavlick, “Does data augmentation improve generalization in nlp?,” *arXiv preprint arXiv:2004.15012*, 2020.
- [2] S. Y. Feng, V. Gangal, J. Wei, S. Chandar, S. Vosoughi, T. Mitamura, and E. Hovy, “A survey of data augmentation approaches for nlp,” *arXiv preprint arXiv:2105.03075*, 2021.
- [3] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.

- [4] Y. Fan, A. Kukleva, D. Dai, and B. Schiele, “Revisiting consistency regularization for semi-supervised learning,” *International Journal of Computer Vision*, vol. 131, no. 3, pp. 626–643, 2023.
- [5] J. M. Joyce, “Kullback-leibler divergence,” in *International encyclopedia of statistical science*, pp. 720–722, Springer, 2011.